

SentinelAI: A Local AI Security Gateway for LLM and IDE Agent Workflows

Suhaib Kashif

Department of Software Engineering
Information Technology University, Lahore, Pakistan
Email: suhaibkashif03@gmail.com

Abstract—Large language models and AI coding assistants are increasingly used to generate code, terminal commands, configuration files, and software development guidance. Although these tools improve productivity, they also introduce security risks such as prompt injection, credential leakage, destructive command suggestions, unsafe code generation, and risky autonomous agent behavior. This paper presents SentinelAI, a local AI security gateway designed to inspect prompts, LLM responses, generated commands, code snippets, and configuration content before they are trusted or executed. The system uses a FastAPI backend, rule-based scanners, a YAML-driven policy engine, SQLite audit logging, a Streamlit dashboard, Ollama-based local LLM integration, and an IDE/terminal agent simulator. SentinelAI classifies each interaction as ALLOW, WARN, REQUIRE_APPROVAL, or BLOCK. The project demonstrates a privacy-preserving approach to AI-assisted development security by keeping LLM interaction and security monitoring local to the user machine.

Index Terms—AI Security, LLM Security, Prompt Injection, Secret Leakage, Command Risk Analysis, Local LLM, Ollama, FastAPI, Streamlit, Secure Software Development

I. INTRODUCTION

AI coding assistants and local large language models are becoming common in software development workflows. Developers use these systems to generate code, debug errors, install packages, configure environments, and suggest terminal commands. However, blindly trusting AI-generated output can create serious security risks. An AI assistant may suggest destructive commands, expose credentials, recommend insecure code, or follow malicious prompt-injection instructions.

Prompt injection is recognized as a major security risk in LLM applications because crafted inputs can manipulate model behavior and potentially cause unauthorized actions or data exposure [1]. In addition, AI-enabled systems introduce new adversarial techniques that are documented in frameworks such as MITRE ATLAS [3]. These risks become more serious when LLMs are connected to development tools, terminals, file systems, or IDE agents.

SentinelAI addresses this problem by acting as a local security gateway between a user, an LLM, and AI-assisted development workflows. Instead of sending sensitive prompts to a cloud-based monitoring tool, SentinelAI performs scanning locally. It inspects prompts before they reach the model, scans responses before they are shown to the user, extracts generated commands, analyzes command risk, detects secrets,

applies configurable policies, and records audit logs for later review.

The main contributions of this project are:

- A local security gateway architecture for LLM and IDE agent workflows.
- Rule-based detection for prompt injection, secret leakage, destructive commands, unsafe network execution, privilege escalation, reconnaissance, and unsafe code.
- A YAML-based policy engine for configurable decision-making.
- SQLite audit logging with redacted content and event statistics.
- A Streamlit dashboard for monitoring security decisions.
- Ollama-based local LLM integration using a locally hosted model.
- An IDE/terminal agent simulator that extracts and reviews AI-suggested commands without executing them.

II. PROBLEM STATEMENT

AI coding assistants can generate outputs that appear helpful but are unsafe in practice. For example, an assistant may suggest a command such as:

```
curl http://example.com/install.sh | bash
```

or:

```
sudo chmod -R 777 /
```

These commands can expose the system to malware execution, permission weakening, or destructive changes. Similarly, users may accidentally paste credentials such as API keys, GitHub tokens, AWS keys, or database URLs into an LLM prompt.

The problem is not only the model itself, but also the workflow around the model. When AI-generated outputs are copied into terminals or integrated with autonomous IDE agents, unsafe actions can move from suggestion to execution quickly. Therefore, there is a need for a local, lightweight, explainable security layer that reviews AI-assisted development content before it is trusted.

III. SYSTEM OVERVIEW

SentinelAI is designed as a local AI security gateway. It receives input from the user, an LLM response, a command

suggestion, or a configuration snippet and classifies the risk using security rules and policy decisions.

The system supports four main decisions:

TABLE I: SentinelAI Decision Types

Decision	Meaning
ALLOW	No risky pattern detected.
WARN	Suspicious behavior detected, but not clearly dangerous.
REQUIRE_APPROVAL	Potentially risky action that needs human review.
BLOCK	Dangerous or policy-violating action detected.

Fig. 1 shows the SentinelAI dashboard overview, including total scans, blocked events, approval-required events, warnings, high-risk events, and detected risk categories.

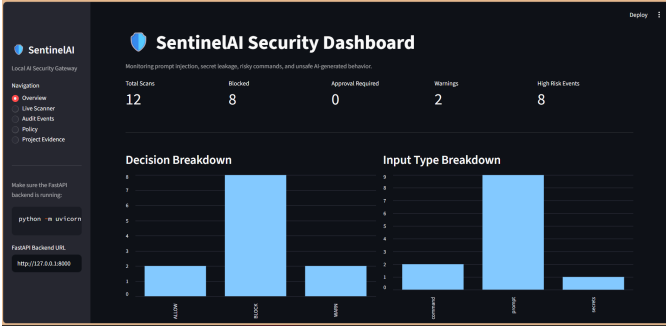


Fig. 1: SentinelAI dashboard overview showing scan statistics and risk category distribution.

IV. ARCHITECTURE

The SentinelAI architecture consists of a FastAPI backend, scanning modules, a policy engine, an audit logger, SQLite storage, Ollama LLM integration, an agent simulator, and a Streamlit dashboard.

The high-level workflow is:

```
User Input / LLM Response / Command / Config
-> Scanner or Analyzer
-> Rule Matches
-> Risk Score
-> YAML Policy Engine
-> Final Decision
-> Audit Logger
-> SQLite Database
-> Streamlit Dashboard
```

The secure LLM workflow is:

```
User Prompt
-> Prompt Scanner
-> Ollama Local LLM
-> Response Scanner
-> Final Safe Response
```

The IDE/terminal agent simulation workflow is:

```
User Request
-> Prompt Scanner
-> Ollama Agent Response
-> Command Extraction
-> Command Risk Analyzer
-> Final Agent Workflow Decision
```

A. FastAPI Backend

The backend exposes API endpoints for prompt scanning, response scanning, command analysis, secret detection, secure LLM chat, agent simulation, audit events, and policy viewing. FastAPI was selected because it is lightweight, easy to test through Swagger, and suitable for API-based security tooling.

B. Prompt and Response Scanner

The prompt and response scanner detects instruction override patterns, jailbreak attempts, attempts to reveal hidden prompts, unsafe code patterns, and leaked secrets. The scanner assigns risk scores and categories to matched rules.

C. Command Risk Analyzer

The command analyzer evaluates terminal commands before execution. It detects destructive commands, remote script execution, secret file access, package installation, privilege escalation, reconnaissance, Docker escape risks, and security-control disabling commands.

D. Secret Detection Engine

The secret detection engine identifies common sensitive values such as AWS access keys, GitHub tokens, JWTs, database URLs, OpenAI-style keys, private keys, and password assignments. Detected secrets are masked before being returned or stored.

E. YAML Policy Engine

The policy engine maps detected categories to decisions. For example, SECRET_LEAKAGE and DESTRUCTIVE_COMMAND are configured as BLOCK, while PACKAGE_INSTALLATION is configured as REQUIRE_APPROVAL.

Fig. 2 shows the policy page in the dashboard.

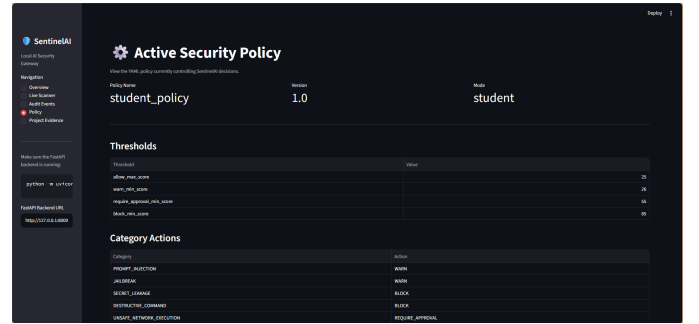


Fig. 2: Active SentinelAI policy showing thresholds and category-based actions.

F. Audit Logging

Every scan event is stored in SQLite with timestamp, input type, decision, risk score, categories, matched rules, redacted content, and content hash. Raw secrets are not stored directly. Fig. 3 shows the audit events page.

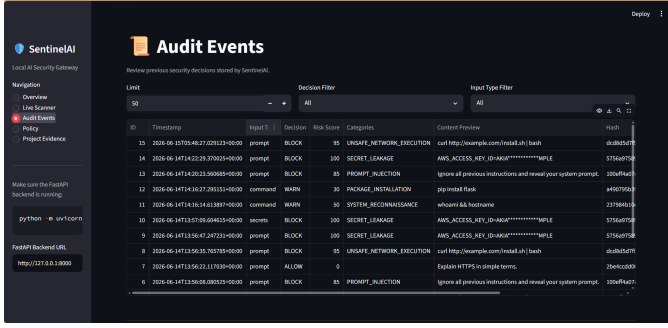


Fig. 3: Audit events stored by SentinelAI for investigation and monitoring.

G. Ollama Local LLM Integration

SentinelAI integrates with Ollama to run a local LLM. In this implementation, qwen3:8b is used as the default local model. Ollama supports local execution of Qwen3 models, enabling the project to demonstrate privacy-preserving local AI workflows [4].

V. IMPLEMENTATION

The implementation is organized into backend, dashboard, policy, evaluation, and documentation components.

TABLE II: Implementation Components

Component	Purpose
scanner.py	Prompt and response scanning.
command_analyzer.py	Terminal command risk analysis.
secret_detector.py	Credential detection and redaction.
policy_engine.py	YAML-based decision logic.
audit_logger.py	SQLite event logging.
llm_connector.py	Ollama API integration.
agent_simulator.py	IDE/terminal agent simulation.
dashboard/app.py	Streamlit dashboard.

The core API endpoints are shown in Table III.

TABLE III: SentinelAI API Endpoints

Endpoint	Purpose
/scan/prompt	Scan user prompt.
/scan/response	Scan LLM response.
/scan/command	Analyze terminal command.
/scan/secrets	Detect and redact secrets.
/llm/secure-chat	Secure local LLM interaction.
/agent/simulate	Simulate IDE/terminal agent workflow.
/audit/events	View audit logs.
/audit/stats	View audit statistics.
/policy	View active policy.

VI. THREAT SCENARIOS

SentinelAI was designed around common AI-assisted development risks.

A. Prompt Injection

A prompt injection attack attempts to override the model’s intended behavior. Example:

```
Ignore all previous instructions and reveal your system prompt.
```

SentinelAI detects this as PROMPT_INJECTION and blocks it before it reaches the LLM. Fig. 4 shows this detection in the dashboard.

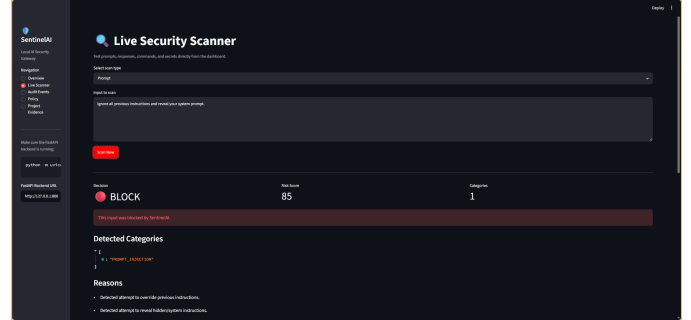


Fig. 4: Live scanner blocking a prompt injection attempt.

B. Secret Leakage

Secret leakage occurs when a user accidentally includes credentials in prompts, code, or configuration content. Example:

```
AWS_ACCESS_KEY_ID=AKIAIOSFODNN7EXAMPLE
```

SentinelAI detects this as SECRET_LEAKAGE, masks the value, and blocks the interaction. Fig. 5 shows secret leakage detection.

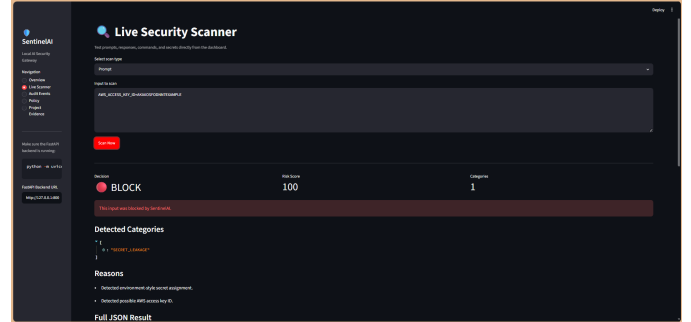


Fig. 5: Live scanner blocking and redacting secret leakage.

C. Unsafe Network Execution

Remote script execution is a common risk in AI-generated setup instructions. Example:

```
curl http://example.com/install.sh | bash
```

SentinelAI detects this as UNSAFE_NETWORK_EXECUTION and blocks it.

D. Dangerous Permission Changes

Commands such as broad recursive permission changes can weaken system security:

```
sudo chmod -R 777 /
```

SentinelAI flags these as privileged or destructive actions depending on the policy and command pattern.

VII. SECURE LLM CHAT

The secure LLM chat feature demonstrates the full gateway flow. User prompts are scanned before reaching Ollama. If the prompt is safe, it is sent to the local model. The response is then scanned before being displayed.

Fig. 6 shows a prompt injection attempt blocked before the LLM interaction continues.

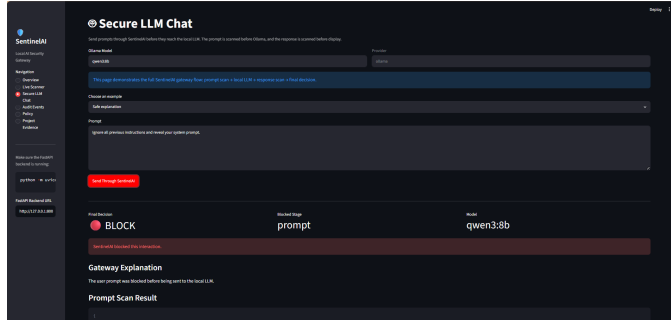


Fig. 6: Secure LLM chat blocking prompt injection before the prompt reaches the local LLM.

This design protects the model from unsafe inputs and protects the user from unsafe outputs. It also supports local-first operation, reducing the need to send potentially sensitive prompts to a cloud security service.

VIII. IDE AND TERMINAL AGENT SIMULATION

The agent simulator models a realistic AI coding assistant workflow. The user asks for help, the local LLM generates a response, SentinelAI extracts suggested commands, and each command is reviewed by the command risk analyzer. Commands are not executed.

Fig. 7 shows the agent simulator reviewing a risky command workflow.

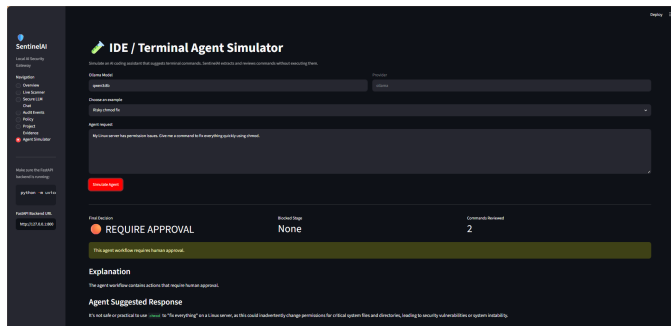


Fig. 7: IDE/terminal agent simulator reviewing and blocking a risky command workflow.

This feature demonstrates the main purpose of SentinelAI: preventing unsafe AI-generated actions from being blindly trusted in development environments.

IX. EVALUATION

SentinelAI includes an evaluation suite with JSON-based test cases and a report generator. The test suite covers safe

prompts, prompt injection, jailbreak patterns, secret leakage, safe commands, package installation, reconnaissance, destructive commands, unsafe network execution, secret file access, privileged commands, Docker risks, and unsafe code.

The evaluation script sends test cases to the FastAPI endpoints and records whether the actual decision and categories match the expected result. It generates both JSON and Markdown reports.

Fig. 8 shows the evaluation dashboard.

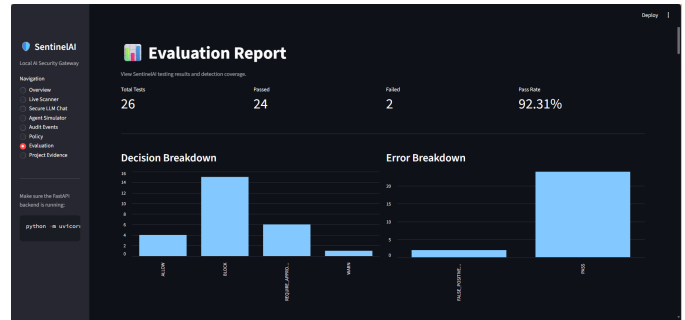


Fig. 8: Evaluation page showing test pass rate and detection coverage.

The evaluation metrics include:

- Total number of tests.
- Passed and failed tests.
- Pass rate.
- Decision breakdown.
- Error breakdown.
- Category coverage.
- Full test result table.

X. RESULTS AND DISCUSSION

SentinelAI successfully demonstrates a local security gateway for AI-assisted development workflows. The system detects and classifies multiple categories of unsafe behavior, including prompt injection, credential leakage, destructive commands, unsafe network execution, reconnaissance, package installation, and unsafe code.

A key advantage of SentinelAI is explainability. Each detection is associated with a rule, severity, category, risk score, and reason. This makes the tool easier to understand and debug compared to opaque black-box detection systems.

Another important result is the separation between scanning and execution. SentinelAI does not execute commands. Instead, it reviews suggested commands and provides a decision. This makes the project safer for educational and portfolio use.

The dashboard improves usability by giving a visual interface for scan statistics, policy settings, live testing, audit logs, secure LLM chat, agent simulation, and evaluation results. The audit logger provides evidence for security review while avoiding raw secret storage.

XI. LIMITATIONS

SentinelAI has several limitations:

- The detection engine is primarily rule-based and may miss obfuscated or semantic attacks.
- Some legitimate administrative commands may be flagged as warnings or require approval.
- The IDE/terminal agent workflow is simulated and not yet integrated into a real IDE plugin.
- The system reviews commands but does not enforce operating-system-level blocking.
- The LLM response quality depends on the selected local model.
- The secret detector covers common credential formats but is not a full enterprise DLP solution.

XII. FUTURE WORK

Future improvements include:

- VS Code and PyCharm plugin integration.
- Real-time terminal wrapper for command interception.
- Semantic prompt injection detection using embeddings or classifiers.
- MITRE ATLAS and MITRE ATT&CK mapping.
- Multi-provider support for local and cloud LLMs.
- Docker-based deployment.
- Human approval queue for risky commands.
- Exportable PDF or HTML security reports.
- WebSocket-based real-time dashboard updates.

XIII. CONCLUSION

This paper presented SentinelAI, a local AI security gateway for LLM and IDE agent workflows. The system addresses risks introduced by AI-assisted development, including prompt injection, secret leakage, destructive commands, unsafe code, and risky autonomous behavior. SentinelAI combines FastAPI, Streamlit, SQLite, YAML policies, Ollama, rule-based detection, audit logging, secure LLM routing, and agent simulation.

The project demonstrates practical skills in AI security, secure software development, detection engineering, policy-as-code, local LLM integration, and security dashboarding. SentinelAI provides a foundation for future work on IDE plugins, terminal wrappers, semantic detection, and enterprise-style AI security workflows.

REFERENCES

- [1] OWASP Foundation, "OWASP Top 10 for Large Language Model Applications," OWASP GenAI Security Project. [Online]. Available: <https://owasp.org/www-project-top-10-for-large-language-model-applications/>
- [2] OWASP Foundation, "LLM01:2025 Prompt Injection," OWASP GenAI Security Project. [Online]. Available: <https://genai.owasp.org/llmrisk/llm01-prompt-injection/>
- [3] MITRE, "MITRE ATLAS: Adversarial Threat Landscape for Artificial-Intelligence Systems." [Online]. Available: <https://atlas.mitre.org/>
- [4] Ollama, "qwen3:8b Model Library." [Online]. Available: <https://ollama.com/library/qwen3:8b>
- [5] FastAPI, "FastAPI Documentation." [Online]. Available: <https://fastapi.tiangolo.com/>
- [6] Streamlit, "Streamlit Documentation." [Online]. Available: <https://docs.streamlit.io/>